

for loop :

It is also **entry-controlled loop**, it is used to write concise code that means everything initialization, condition, and increment/decrement written in a single line.

It is preferable when number of iterations are **fixed**.

Syntax: **1** **2** **4**

```
for (initialization; condition; increment/decrement)
{
    // statements 3
}
```

For1.java

```
-----
/* Print Welcome to Java 10 times */

void main()
{
    for(int i=1; i<=10; i++)
    {
        IO.println("Welcome to Java");
    }
}
```

For2.java

```
-----
/* WAP print the number from 1 to n based on the user choice */

void main()
{
    int num = Integer.parseInt(IO.readLine("Enter a number :"));

    for(int i=1; i<=num; i++)
    {
        IO.print(i+" ");

        if(i%5==0)
        {
            IO.println();
        }
    }
}
```

For3.java

```
-----
/* WAP print all the even numbers from 1 to 100 */

void main()
{
    for(int i=2; i<=100; i=i+2)
    {
        IO.print(i+" ");
    }
}
```

For4.java

```
-----
/* WAP print sum of first 100 natural number 1+2+3+4.....100 */

void main()
{
    int sum = 0;
    for(int i=1; i<=100; i++)
    {
        sum += i;
    }
    IO.println("Sum of first 100 natural number is :"+sum);
}
```

For5.java

```
-----
//use of break

void main()
{
    for(int i=1; i<=10; i++)
    {
        if(i==5)
            break;

        IO.println(i);
    }
}
```

For6.java

```
-----
//use of continue

void main()
{
    for(int i=1; i<=10; i++)
    {
        if(i==5)
            continue; //Will skip the current iteration of loop

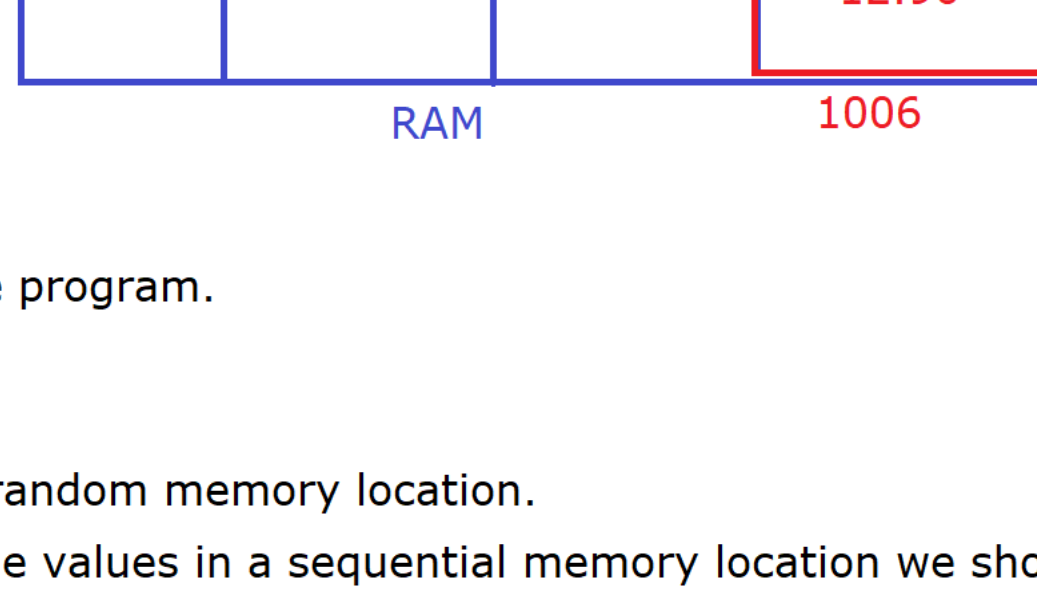
        IO.println(i);
    }
}
```

Variable :

- * It is a name given for the memory location.
- * It is used to hold some meaningful value.

Example : int x = 10;
float y = 12.90f;
x = x + 30;

VARIABLE
VARY + ABLE
CHANGE + ABLE



* A variable can change its value during the execution of the program.

Drawback of an ordinary Variable :

- 1) An ordinary variable can hold only one value at a time in random memory location.
Example : int x = 10,20; //Invalid [In order to store multiple values in a sequential memory location we should use array.]
- 2) Once the program execution is completed we will not get back the value of the variable because they are stored in RAM i.e volatile memory. [File Handling, Here we will store the data in a file]

For each loop :

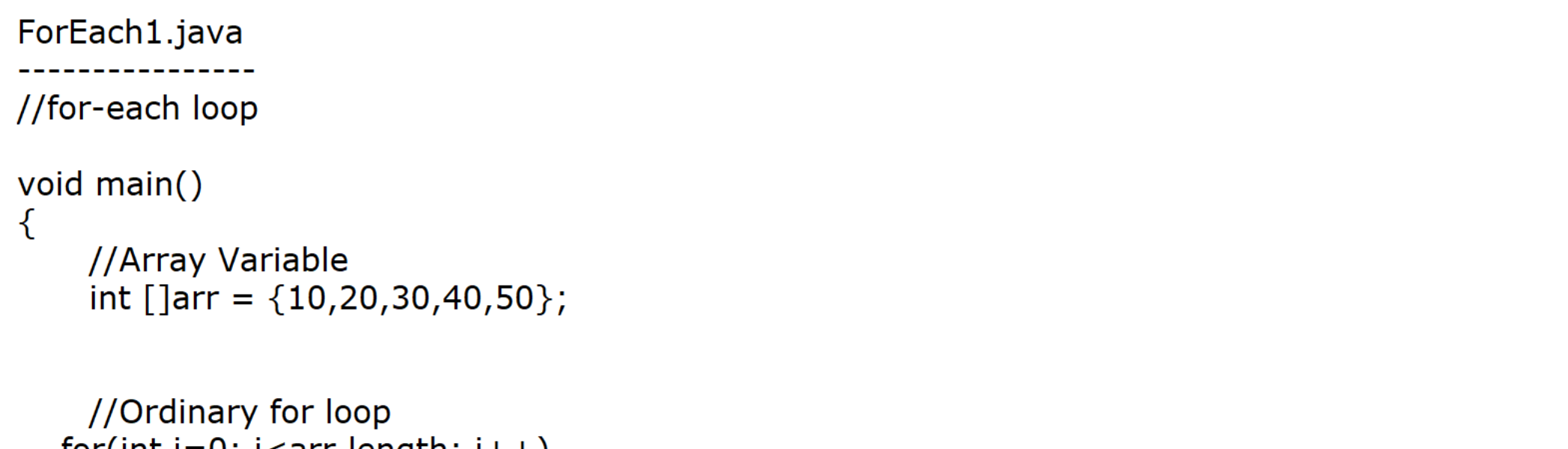
* It is an **enhanced for loop** which was introduced from JDK 1.5V.

* It is used to **fetch OR retrieve** the elements one by one from the **array OR collection**.

* Actually Java compiler will generate an ordinary for loop for this for each loop.

Syntax :

```
for(data type variable : array variable)
{
    //body of the for each loop
}
```



ForEach1.java

```
-----
//for-each loop

void main()
{
    //Array Variable
    int []jarr = {10,20,30,40,50};

    //Ordinary for loop
    for(int i=0; i<arr.length; i++)
    {
        IO.println(arr[i]);
    }

    IO.println(".....");

    //for each loop
    for(int x : arr)
    {
        IO.println(x);
    }
}
```

ForEach2.java

```
-----
//for-each loop

void main()
{
    double []values = {12.90, 56.34, 23.90, 45.78};

    for(double value : values)
    {
        IO.println(value);
    }
}
```

values is an array variable which can hold multiple values, on the other hand value is an ordinary variable so, It can hold only one value for each iteration.

ForEach3.java

```
-----
//for-each loop

void main()
{
    String []colors = {"White", "Blue", "Green", "Pink", "Red"};

    for(String color : colors)
    {
        IO.println(color);
    }
}
```

ForEach4.java

```
-----
//for-each loop

void main()
{
    Object []values = {12, 45.89, 'A', true, "Java"};

    for(Object value : values)
    {
        IO.println(value);
    }
}
```